```
        std::future<std::list<T> > new_lower=                    ❸
            pool.submit(std::bind(&sorter::do_sort,this,
                                  std::move(new_lower_chunk)));

        std::list<T> new_higher(do_sort(chunk_data));

        result.splice(result.end(),new_higher);
        while(!new_lower.wait_for(std::chrono::seconds(0)) ==
            std::future_status::timeout)
        {
            pool.run_pending_task();                             ❹
        }

        result.splice(result.begin(),new_lower.get());
        return result;
    }
};

template<typename T>
std::list<T> parallel_quick_sort(std::list<T> input)
{
    if(input.empty())
    {
        return input;
    }
    sorter<T> s;

    return s.do_sort(input);
}
```

Just as in listing 8.1, you've delegated the real work to the `do_sort()` member function of the `sorter` class template ❶, although in this case the class is only there to wrap the `thread_pool` instance ❷.

Your thread and task management is now reduced to submitting a task to the pool ❸ and running pending tasks while waiting ❹. This is much simpler than in listing 8.1, where you had to explicitly manage the threads and the stack of chunks to sort. When submitting the task to the pool, you use `std::bind()` to bind the `this` pointer to `do_sort()` and to supply the chunk to sort. In this case, you call `std::move()` on the `new_lower_chunk` as you pass it in, to ensure that the data is moved rather than copied.

Although this has now addressed the crucial deadlock-causing problem with tasks that wait for other tasks, this thread pool is still far from ideal. For